FIT2109 Computer Science Workshop

Week 2: Shell Tools & Scripting

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

From Week 1 to reliable automation

Week 1 gave us the control surface

We can navigate, inspect files, run commands, read permissions, and reason about streams.

This week is about precision

This week focuses on what each command actually receives. Quoting, variables, globs, exit status, and safety flags decide whether automation is reliable or fragile.

Learning goals

By the end of this workshop, you should be able to:

- Predict how quoting, variables, globs, and splitting affect a command.
- Use `export`, `env`, `$?`, `&&`, and `||`.
- Explain how the shell expands globs to match filenames.
- Build `grep`, `find`, `sort`, `uniq`, and `wc` pipelines.
- Review and improve Bash scripts using arguments, conditionals, loops, simple functions, and safe quoting.
- Connect to a remote server with `ssh`, transfer files with `scp`, and explain why SSH keys replace passwords.

Case study: investigate overnight errors

The real task

Several services wrote log files overnight. Produce one report that counts each error code, including errors in archived subdirectories.

```
log-report-example/
|-- log-report.sh
`-- logs/
    |-- app.log
    |-- api.log
    `-- archive/
        `-- old.log
```

Today's example

The file `log-report.sh` will turn these raw logs into a ranked error report. We will explain the script one part at a time.

Before running a command, the shell prepares its arguments

From typed text to arguments

A program receives a command name and a list of arguments, not the exact text you typed. The shell uses quotes, variables, and filename patterns to prepare that list.

In `log-report.sh`

```
find "$log_dir" -type f -name '*.log'
```

- The shell expands `$log_dir` to `logs` and removes the quote characters.
- `find` receives separate arguments including `logs`, `-type`, `f`, `-name`, and the literal pattern `*.log`.

Question to ask

What arguments does the command actually receive?

Quoting controls what the shell changes

No quotes

Variables expand, then split on spaces.

```
file="Week 2.txt"
cat $file
```

Shell passes two filenames: `Week` and `2.txt`

Double quotes

Variables expand, but spaces stay inside one argument.

```
cat "$file"
```

Shell passes one filename: `Week 2.txt`

Single quotes

Text is literal. No variable expansion.

```
cat '$file'
```

Shell looks for a file named: `$file`

In `log-report.sh`

`"$log_dir"` follows the double-quote rule: expand the variable, but keep its value as one argument. Make this a habit even when today's value, `logs`, has no spaces.

A shell can start child processes

A process is a running program

Your current shell is already running. When it starts another program, the shell is the **parent process** and the new running program is its **child process**.

Parent process

Current shell

```
$ bash log-report.sh
```

Child processes in the script

```
find   grep
sort   uniq
```

What the child sends back

Each child writes output and returns an exit status. The shell running `log-report.sh` starts these programs and connects their streams with pipes.

Shell variable: the current shell prepares an argument

In `log-report.sh`

```
log_dir=logs
find "$log_dir" -type f -name '*.log' ...
```

Parent shell

Stores `log_dir=logs`

Expands `"$log_dir"`

Child `find`

Receives the argument `logs`

Does not receive a variable named `log_dir`

No export is needed here

The parent shell replaces `$log_dir` before it starts `find`.

Environment variable: `export` passes a setting to a child

In the parent shell

Define two differently named variables:

```
log_dir=logs
export CHILD_NOTE="passed"
```

Then start a child shell:

```
bash
```

Inside the child Bash

```
echo "$log_dir"
```

prints a blank line.

```
echo "$CHILD_NOTE"
```

prints `passed`.

The difference

The ordinary shell variable stays in the parent. The exported variable enters the environment inherited by the child.

Demo 1: shell evaluation

Watch

Compare unquoted and quoted variables, then start a child Bash and see which of two differently named variables it inherits.

Question to keep asking

What arguments and settings does the child process actually receive?

Exit status lets commands make decisions

Key idea

Every command returns an exit status. 0 usually means success; nonzero means failure.

- `$?`: exit status of the previous command.
- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeds.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` fails.

Ask a second question about the same data

```
grep -q CRITICAL logs/app.log && echo 'critical error found'
```

`grep -q` prints nothing: its exit status answers whether a match exists.

Demo 2: exit status as control flow

Watch

Use `$?`, `&&`, and `||` to react to searches in the same `logs/` folder.

Pause and reason

Why can `grep -q` answer yes or no without printing the matching line?

A glob is a filename pattern

What happens

A glob is a pattern that the shell expands into matching filenames *before* it runs the command.

Inside the example's `logs/` directory

`app.log` `api.log` `archive/old.log`

`ls logs/*.log` $\xrightarrow{\text{shell expands the glob}}$ `ls logs/app.log logs/api.log`

Why `log-report.sh` later uses `find`

The shell glob matches direct children only, so it misses `logs/archive/old.log`. The script uses `find` later because the real task includes subdirectories.

`*` matches any sequence of characters; `?` matches one character; `[0-9]` matches one digit.

grep searches text

Basic form

`grep` *PATTERN FILE*

`grep` reads the file line by line and prints each line that matches the pattern. The italic words are placeholders that you replace.

Start with one file from the example folder

```
grep ERROR logs/app.log
```

- The pattern can be simple text such as `ERROR`.
- Matching lines go to standard output, so they can be redirected or piped.
- `log-report.sh` refines the same command to extract error codes from several files.

Meet `sort`, `uniq`, and `wc`

`sort`: order lines

In `log-report.sh`, `sort` places equal error codes next to each other.

`uniq`: combine adjacent duplicates

The script then uses `uniq -c` to collapse adjacent equal codes and prefix each code with its count.

`wc`: count text

While building the script, `grep ERROR logs/app.log | wc -l` answers the simpler question "how many error lines are in this file?"

Text tools compose into a pipeline

Follow the pipes near the bottom of `log-report.sh`

```
grep output | sort | uniq -c | sort -nr
```

1. The earlier `find ... -exec grep ...` stage produces one error code per line.
2. `sort` places equal lines together.
3. `uniq -c` combines adjacent duplicates and adds counts.
4. `sort -nr` puts the largest numeric count first.

grep flags refine the search

Used in `log-report.sh`

- `-h`: omit filenames when searching multiple files.
- `-E`: use extended regular-expression syntax.
- `-o`: print only the matched text.

Useful for other questions

- `-n`: show line numbers.
- `-q`: print nothing; use exit status.
- `-r`: search directories recursively.

In `log-report.sh`

```
grep -hEo 'ERROR [0-9]+'
```

Together, these flags remove filename prefixes, enable +, and print only text such as `ERROR 404`.

Demo 3: build the text-processing pipeline

Watch

Start with `grep ERROR logs/app.log`, introduce `wc -l`, then add `sort`, `uniq -c`, and the exact `grep` flags used by `log-report.sh`.

Limitation to notice

The final `logs/*.log` glob still misses `logs/archive/old.log`. The next concept fixes that problem.

`find`: basic file search

Basic form

`find` *PATH* `-type f -name` *PATTERN*

PATH says where to search; *PATTERN* says which filenames to keep.

Find every log file below `logs`

```
find "$log_dir" -type f -name '*.log'
```

`"$log_dir"` Expand the script's variable and start searching there.

`-type f` Keep regular files, not directories.

`-name '*.log'` Keep filenames ending in `.log`.

Why quote `'*.log'`?

`find` must receive this pattern itself. Quoting prevents early shell expansion, so `find` can also reach `logs/archive/old.log`.

find: add a command to the basic search

The two continued lines in log-report.sh

```
find "$log_dir" -type f -name '*.log'
  -exec grep -hEo 'ERROR [0-9]+' {} +
```

`-exec` Run a command using the paths that `find` matched.

`{}` The place where the matching paths are inserted.

`+` Pass several paths to one command when possible.

Read it left to right

First find the files; then give their paths to `grep`. Its extracted error codes flow into the pipe after +.

Demo 4: extend the report with `find`

Watch

Compare `logs/*.log` with `find logs -type f -name '*.log'`, add `-exec grep ... {} +`, then run the complete `log-report.sh`.

Question to answer

Which part makes `logs/archive/old.log` enter the report?

Run `log-report.sh` and trace the data

```
$ bash log-report.sh
      3 ERROR 404
      2 ERROR 500
      1 ERROR 503
      1 ERROR 403
```

1. `logs/` supplies files, including `archive/old.log`.
2. `find` supplies paths; `grep` turns file contents into error codes.
3. `sort | uniq -c | sort -nr` turns codes into a ranked report.

Experiment

Run one stage at a time, or temporarily remove a stage, and observe how the output changes.

Handout Activity 1: log investigation

Now work on Activity 1 in the handout

Your group has a server log and real questions to answer: which errors occur, and which occur most often? Design the commands, build the pipeline, and defend each stage.

Group output

Complete Parts A–B: answer the log questions with `grep` and a counting pipeline, then track down files with `find`.

Scripts make workflows reusable

Key idea

A shell script is a text file that Bash reads and evaluates, just as if you typed each command yourself.

- Shebang: `#!/usr/bin/env bash`
  macOS users: this ensures Bash runs the script even if your interactive shell is Zsh.
- Positional arguments: `$1`, `$2`, ...
- Argument count: `$#` — check it first: `[ "$#" -eq 0 ]`
- Default value: `${1:-world}`
- Conditional: `if`, `then`, `else`, `fi`
- File tests: `[ -f "$file" ]` for a file, `[ -d "$dir" ]` for a directory.
- Loop: `for x in ...; do ...; done`

Bash can group commands into functions

Recognition pattern

A Bash function names a small command sequence so the script can call it more than once.

```
warn() {
  echo "warning: $1" >&2
}

warn "missing input"
```

What does $1 mean?

In a Bash function, `$1` is the first argument passed to the function call. Here, `warn "missing input"` makes `$1` equal to `missing input`.

Why shell-script reliability matters

Evidence from real projects

A study of real open-source Bash scripts found recurring bugs involving quoting, word splitting, path management, command options, return values, and error handling.

Connection to this workshop

Quoted variables, checked exit statuses, explicit paths, `set -u`, and `pipefail` directly address these common failure modes.

Research reference

Bash in the Wild: Language Usage, Code Smells, and Bugs — Yiwen Dong, Zheyang Li, Yongqiang Tian, et al. ACM Transactions on Software Engineering and Methodology (TOSEM), 2022.

Reliable scripts fail clearly

Three safety habits

- `set -u`: fail on unset variables.
- `set -o pipefail`: fail if any pipeline stage fails.
- Quote variables: `"$1"`, `"$file"`.

Why this matters

Silent empty strings and hidden pipeline failures can make a script appear successful when it did the wrong thing.

Useful debugging trace

`bash -x script.sh` or `set -x` prints each command after expansion, which is handy for showing what the shell actually runs. ShellCheck (`shellcheck.net`) reviews scripts for these pitfalls automatically.

Demos 5–6: script mechanics and silent failures

Watch

Demo 5: make a small script executable and pass it arguments. Demo 6: a script fails *silently* — expose the bug with `bash -x`, then make it fail loudly with `set -u`.

Note

The demo shows how to *find* a hidden failure. In Activity 2, your group finds and fixes the failures in a different script.

Demo 6 continued: loop over log files

Watch

Turn a one-file command into a reusable loop over many log files.

Pause and reason

If no `.log` file matches the glob, what does the loop variable receive — and what happens then?

Handout Activity 2: script review and repair

Today's main group task (about 25 minutes)

Review a fragile script, identify failure modes, and rewrite it using safer Bash habits — this is where everything from today comes together.

Group output

Complete Parts A–D: bug list, safer rewrite, safety-flag explanation, and loop design.

What is SSH?

SSH means Secure Shell

SSH is a protocol for securely connecting to another machine over a network. It encrypts the connection so you can log in, run commands, and transfer files safely.

- It is the standard way to work with remote Linux servers.
- It supports both interactive sessions and one-off remote commands.
- Your local terminal stays the same; the commands run on the other machine.

Logging in with a password

The simplest way in

```
ssh username@fit2109s2.rep.monash.edu
```

- On the first connection, SSH asks you to trust the server's fingerprint.
- Then the server asks for your account password.
- You now have a shell *on the remote machine*; type `exit` to come back.
- `ssh user@host 'command'`: run one remote command.
- `scp local user@host:/path/`: copy to remote.
- `scp user@host:/path/file .`: copy from remote.

SSH key login: the private key never leaves your device

1. Server

Sends a fresh, one-time challenge

→

2. Your device

Private key signs the challenge locally

The key is never sent

→

3. Server

Public key verifies the signature

Valid proof means access

Why this is safer than a password

No reusable password is sent or stored in a script. A public key or captured signature cannot create a new valid signature; forging one with a modern key is computationally infeasible.

The remaining risk — protect the private key

If someone steals your private key, they may impersonate you. Keep the file on a trusted device, restrict its permissions, and add a **passphrase** so the key file is encrypted at rest.

You will set up SSH keys in the Applied session

1. `ssh-keygen -t ed25519`: create a private/public key pair and add a passphrase when prompted.
2. `ssh-copy-id user@host`: copy only the public key to the server.
3. `ssh user@host`: the server challenges your device, then checks its signed proof.

Protect the private key

Copy only the public-key file to the server. Never upload or share the private-key file.

Final checkpoint: Poll Everywhere

pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to pollev.com/fit2109.

What should feel different after today?

- You can explain why quoting changes command behaviour.
- You can use exit status to make shell workflows branch safely.
- You can explain how globs match filenames and use `grep` to search text.
- You can build a pipeline to answer a question from text.
- You can review a script for quoting, missing input, and hidden failures.
- You can describe how SSH keys support remote command-line workflows.

Exit ticket

1. Why do `$file` and `"$file"` sometimes pass a different number of arguments?
2. Before `echo *.log` runs, what does the shell do with `*.log`?
3. What does `cd "$dir" || exit 1` protect a script against?
4. What do `set -u` and `set -o pipefail` each catch?
5. After `ssh-copy-id`, why does `ssh` stop asking for your password?

Reflection: Which Bash habit from today do you most want to make automatic?

Q&A and next steps

Questions?

Before the applied class

Practise explaining not just what a command does, but why it is written that way. On Ed, read and run the worked `log-report` example, then try the `warn-summary` coding challenge.